

# Lab 2 Docker 基础操作实验报告

学生姓名：姜丞骏 GitHub: whatokk

## 一、实验目的

本次实验围绕 Docker 官方入门教程中的容器运行基础操作展开。实验目标是通过实际命令掌握 Docker 容器端口发布、覆盖默认运行参数、数据持久化、本地文件共享以及多容器应用部署等内容，并将主要步骤截图整理为实验报告。

通过本次实验，需要掌握以下内容：

- 1. 理解 Docker 容器、镜像和 Docker Compose 的基本概念。
- 2. 使用 `docker run` 启动容器。
- 3. 使用端口映射将容器服务暴露到宿主机。
- 4. 使用环境变量、命令覆盖等方式修改容器默认行为。
- 5. 使用 Docker volume 实现容器数据持久化。
- 6. 使用 bind mount 将宿主机文件共享给容器。
- 7. 使用 Docker Compose 管理多容器应用。
- 8. 将实验过程截图和说明整理为 Markdown 文档，并提交到 GitHub 仓库。

## 二、实验环境

| 项目     | 内容             |
|--------|----------------|
| 操作系统   | Windows 11     |
| 容器运行环境 | Docker Desktop |
| 命令行工具  | PowerShell     |
| 容器工具   | Docker CLI     |
| 编排工具   | Docker Compose |
| 版本管理工具 | Git            |
| 报告格式   | Markdown       |
| 提交平台   | GitHub         |

本实验使用 Docker Desktop 作为本地容器运行环境，使用 PowerShell 执行 Docker 命令。实验过程中主要使用 `nginx`、`alpine`、`postgres`、`mysql`、`wordpress` 等镜像完成基础验证。

## 三、实验内容概述

本次实验主要参考 Docker 官方 Running containers 相关教程，完成以下内容：

1. Publishing and exposing ports: 发布容器端口。
2. Overriding container defaults: 覆盖容器默认配置。
3. Persisting container data: 持久化容器数据。
4. Sharing local files with containers: 在宿主机和容器之间共享本地文件。
5. Multi-container applications: 使用 Docker Compose 部署多容器应用。

实验整体流程如下：



## 四、实验步骤与结果

### 4.1 检查 Docker Desktop 环境

首先启动 Docker Desktop，并在 PowerShell 中检查 Docker 和 Docker Compose 是否可用。

```
docker version
docker compose version
docker info
```

实验结果如下图所示。

```
(base) PS D:\money\20260507\lab2> docker version
Client:
Version:      28.5.1
API version:  1.51
Go version:   gol.24.8
Git commit:   e180ab8
Built:        Wed Oct  8 12:19:16 2025
OS/Arch:      windows/amd64
Context:      desktop-linux

Server: Docker Desktop 4.48.0 (207573)
Engine:
Version:      28.5.1
API version:  1.51 (minimum version 1.24)
Go version:   gol.24.8
Git commit:   f8215cc
Built:        Wed Oct  8 12:17:24 2025
OS/Arch:      linux/amd64
Experimental: false
containerd:
Version:      1.7.27
GitCommit:    05044ec0a9a75232cad458027ca83437aae3f4da
runc:
Version:      1.2.5
GitCommit:    v1.2.5-0-g59923ef
docker-init:
Version:      0.19.0
GitCommit:    de40ad0
(base) PS D:\money\20260507\lab2> docker compose version
Docker Compose version v2.40.0-desktop.1
(base) PS D:\money\20260507\lab2> docker info
Client:
Version:      28.5.1
Context:      desktop-linux
Debug Mode: false
Plugins:
ai: Docker AI Agent - Ask Gordon (Docker Inc.)
   Version:   v1.9.11
   Path:      C:\Program Files\Docker\cli-plugins\docker-ai.exe
buildx: Docker Buildx (Docker Inc.)
   Version:   v0.29.1-desktop.1
   Path:      C:\Program Files\Docker\cli-plugins\docker-buildx.exe
cloud: Docker Cloud (Docker Inc.)
   Version:   v0.4.39
   Path:      C:\Program Files\Docker\cli-plugins\docker-cloud.exe
compose: Docker Compose (Docker Inc.)
   Version:   v2.40.0-desktop.1
```

#### Docker 环境检查

从截图可以看到，Docker Client、Docker Server 和 Docker Compose 均能正常输出信息，说明 Docker Desktop 已经启动并可以正常使用。

## 4.2 发布容器端口

Docker 容器默认运行在隔离网络环境中。如果希望从宿主机访问容器中的服务，需要使用端口发布功能，将容器端口映射到宿主机端口。

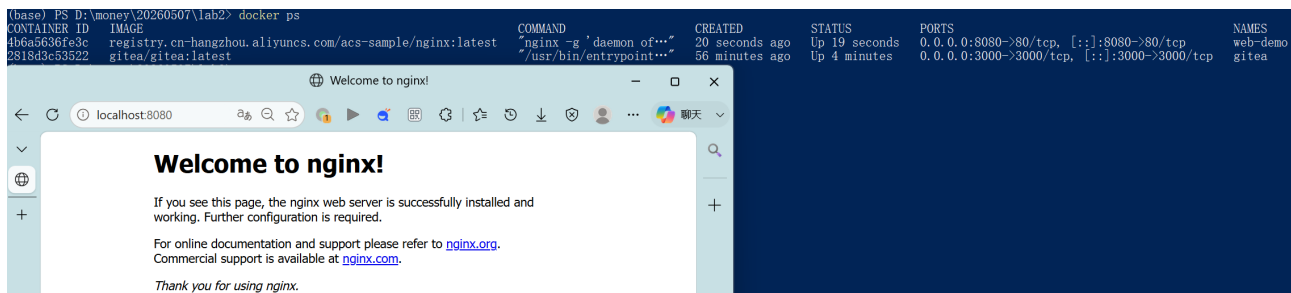
本实验使用 nginx 镜像启动一个 Web 服务，并将容器内的 80 端口映射到宿主机的 8080 端口。

```
docker run -d --name web-demo -p 8080:80 nginx
docker ps
```

在浏览器中访问：

<http://localhost:8080>

实验结果如下图所示。



nginx 端口发布

从截图可以看到，nginx 容器已经成功运行，并且宿主机的 8080 端口已经映射到容器的 80 端口。浏览器能够打开 nginx 默认页面，说明端口发布成功。

清理该容器：

```
docker rm -f web-demo
```

## 4.3 覆盖容器默认配置

Docker 镜像通常带有默认启动命令和默认环境。运行容器时，可以通过 `docker run` 的参数覆盖这些默认行为，例如设置环境变量、修改端口映射、限制资源或覆盖默认命令。

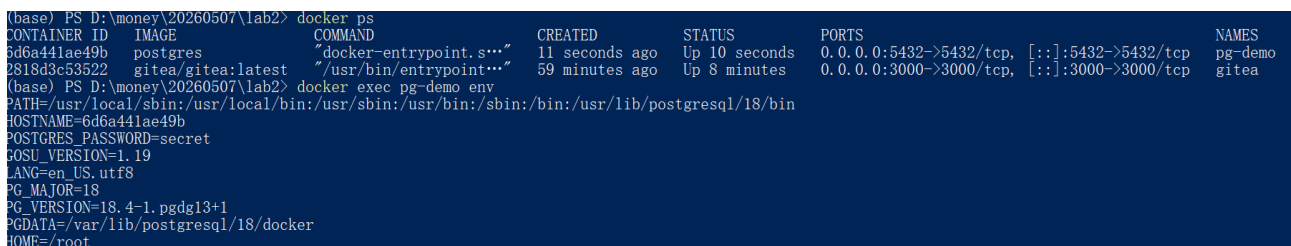
### 4.3.1 使用环境变量启动 Postgres

使用 Postgres 镜像启动数据库容器，并通过环境变量设置数据库密码。

```
docker run -d --name pg-demo \
-e POSTGRES_PASSWORD=secret \
-p 5432:5432 \
postgres
```

```
docker ps
docker exec pg-demo env
```

实验结果如下图所示。



使用环境变量覆盖默认配置

从截图可以看到，Postgres 容器已经成功启动，并且环境变量 `POSTGRES_PASSWORD` 已经传入容器。

清理容器：

```
docker rm -f pg-demo
```

### 4.3.2 覆盖容器默认命令

使用 Alpine 镜像执行自定义命令。

```
docker run --rm alpine echo "Hello Docker";
docker run -it --rm alpine sh
```

进入容器后执行：

```
cat /etc/os-release
exit
```

实验结果如下图所示。



```
(base) PS D:\money\20260507\lab2> docker run --rm alpine echo "Hello Docker"
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
Digest: sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
Status: Downloaded newer image for alpine:latest
Hello Docker
(base) PS D:\money\20260507\lab2> docker run -it --rm alpine sh
/ # cat /etc/os-release
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.23.4
PRETTY_NAME="Alpine Linux v3.23"
HOME_URL="https://alpinelinux.org/"
BUG_REPORT_URL="https://gitlab.alpinelinux.org/alpine/aports/-/issues"
/ # exit
```

覆盖容器默认命令

从截图可以看到，容器没有使用长期运行的默认服务，而是执行指定命令后退出。这说明容器运行时可以覆盖镜像默认命令。

## 4.4 使用 Docker volume 持久化数据

容器的可写层会随着容器删除而丢失。为了让数据在容器删除后仍然保留，可以使用 Docker volume 保存数据。

创建 volume：

```
docker volume create todo-data
docker volume ls
```

使用 Alpine 容器向 volume 写入文件：

```
docker run --rm `
-v todo-data:/data `
alpine sh -c "echo persistent-data &gt; /data/test.txt"
```

删除容器后，重新启动另一个容器读取该文件：

```
docker run --rm `
-v todo-data:/data `
alpine cat /data/test.txt
```

查看 volume 详细信息：

```
docker volume inspect todo-data
```

实验结果如下图所示。

```
(base) PS D:\money\20260507\lab2> docker volume inspect todo-data
[
  {
    "CreatedAt": "2026-05-18T04:31:10Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/todo-data/_data",
    "Name": "todo-data",
    "Options": null,
    "Scope": "local"
  }
]
```

Docker volume 数据持久化

从截图可以看到，即使前一个容器已经删除，写入 volume 的 `test.txt` 文件仍然可以被新容器读取。说明 Docker volume 可以实现容器数据持久化。

实验完成后可以删除 volume：

```
docker volume rm todo-data
```

## 4.5 使用 bind mount 共享本地文件

除了 Docker volume，也可以使用 bind mount 将宿主机目录直接挂载到容器中。该方式常用于开发环境，使容器能够实时访问宿主机上的代码或配置文件。

首先创建本地目录和 HTML 文件：

```
mkdir D:\money\20260507\lab2\site
Set-Content D:\money\20260507\lab2\site\index.html "<h1>Hello from bind mount</h1>";
```

使用 nginx 容器挂载本地目录：

```
docker run -d --name bind-nginx `
  -p 8081:80 `
  -v D:\money\20260507\lab2\site:/usr/share/nginx/html:ro `
  nginx
```

在浏览器中访问：

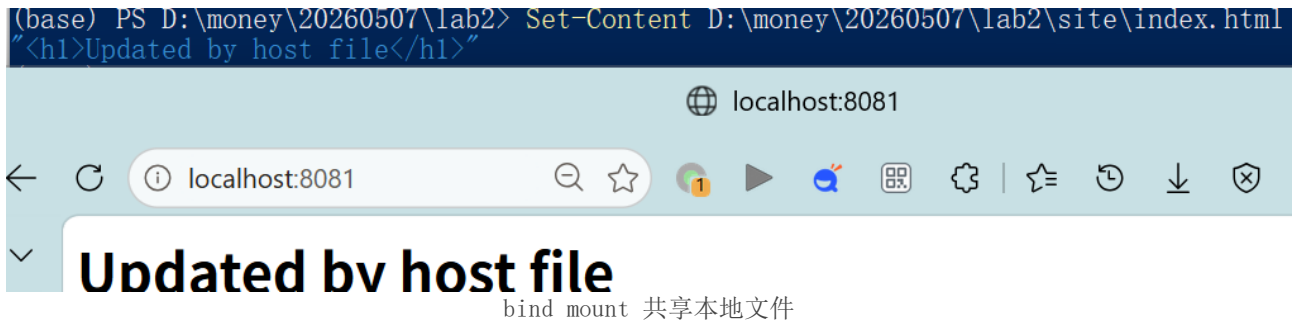
<http://localhost:8081>

修改本地文件：

```
Set-Content D:\money\20260507\lab2\site\index.html "<h1>Updated by host file</h1>";
```

刷新浏览器后，页面内容同步变化。

实验结果如下图所示。



从截图可以看到，浏览器显示了本地 `index.html` 文件中的内容，说明宿主机目录已经成功挂载到容器中。

清理容器：

```
docker rm -f bind-nginx
```

## 4.6 使用 Docker Compose 部署多容器应用

多容器应用通常需要多个服务协同工作。例如，一个 Web 应用可能需要同时运行 Web 服务和数据库服务。Docker Compose 可以通过一个 `compose.yml` 或 `docker-compose.yml` 文件定义多个容器服务。

本实验使用 WordPress 和 MySQL 构建一个简单的多容器应用。

创建实验目录：

```
mkdir D:\money\20260507\lab2\wordpress-demo
cd D:\money\20260507\lab2\wordpress-demo
```

创建 `docker-compose.yml` 文件。

```
notepad docker-compose.yml
```

写入以下内容：

```
services:
  db:
    image: mysql:8.0
    container_name: wordpress-db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: wordpress
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    container_name: wordpress-web
    restart: always
    depends_on:
```

```

- db
ports:
- "8082:80"
environment:
  WORDPRESS_DB_HOST: db:3306
  WORDPRESS_DB_USER: wordpress
  WORDPRESS_DB_PASSWORD: wordpress
  WORDPRESS_DB_NAME: wordpress

```

```

volumes:
  db_data:

```

启动多容器应用：

```

docker compose up -d
docker compose ps
docker ps

```

实验结果如下图所示。

```

(base) PS D:\money\20260507\lab2\wordpress-demo> docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
wordpress-db        mysql:8.0            "docker-entrypoint.s...  db         2 seconds ago  Up 1 second   3306/tcp, 33060/tcp
wordpress-web        wordpress:latest      "docker-entrypoint.s...  wordpress  1 second ago   Up Less than a second  0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
(base) PS D:\money\20260507\lab2\wordpress-demo> docker ps
CONTAINER ID        IMAGE                COMMAND                CREATED        STATUS        PORTS                NAMES
b073daf51a5c        wordpress:latest      "docker-entrypoint.s...  41 seconds ago  Up 40 seconds  0.0.0.0:8082->80/tcp, [::]:8082->80/tcp  wordpress-web
cc1d79c58f81        mysql:8.0            "docker-entrypoint.s...  42 seconds ago  Up 40 seconds  3306/tcp, 33060/tcp   wordpress-db
2818d3c53522        gitea/gitea:latest    "/usr/bin/entrypoint...  About an hour ago  Up 14 minutes  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp  gitea

```

Docker Compose 多容器应用

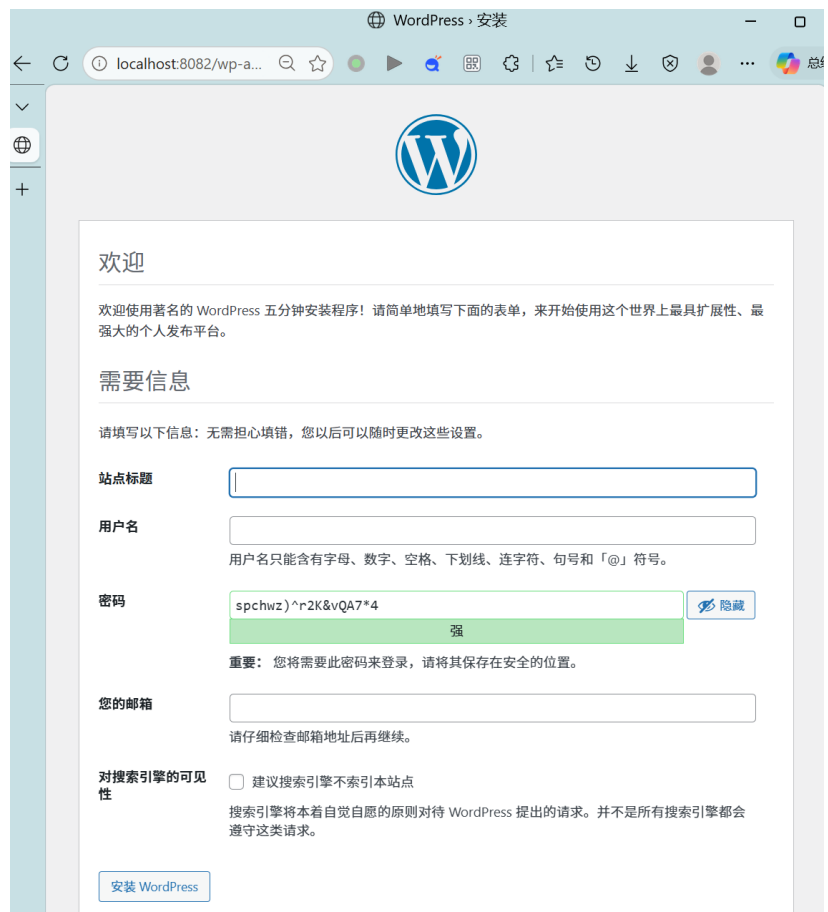
从截图可以看到，MySQL 和 WordPress 两个容器均已启动。WordPress 服务通过 `depends_on` 依赖数据库服务，数据库数据通过 `db_data` volume 持久化保存。

## 4.7 访问 WordPress 应用

在浏览器中访问：

`http://localhost:8082`

实验结果如下图所示。



WordPress 页面访问

从截图可以看到，WordPress 初始化页面已经成功打开，说明多容器应用中的 Web 服务能够正常访问数据库服务。

查看 Compose 创建的网络和 volume：

```
docker network ls
docker volume ls
docker compose ps
```

实验结果如下图所示。

```
(base) PS D:\money\20260507\lab2\wordpress-demo> docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
93bb577d20ec        bridge              bridge              local
c22334249218        host                host                local
ec7956fc5b63        lab1_gitea          bridge              local
78f030d67cad        minikube             bridge              local
049837f7bd9f        none                 null                local
1230d8cdf7ca        wordpress-demo_default bridge              local
(base) PS D:\money\20260507\lab2\wordpress-demo> docker volume ls
DRIVER              VOLUME NAME
local               1c510feb638c0bf1127fcd767190b12e49469bfcae2a79ef1c466e6722d91520
local               75e46eea850106f7a845b094293f2d96ab1070d55f778437e5a18e5886811c6e
local               b8ba9463061b5bc2cf17bde4f00b1998956302b83b64607227f66396d92d42ef
local               minikube
local               wordpress-demo_db_data
(base) PS D:\money\20260507\lab2\wordpress-demo> docker compose ps
NAME                IMAGE                COMMAND              SERVICE    CREATED          STATUS
wordpress-db        mysql:8.0            "docker-entrypoint.s...  db         About a minute ago Up Ab
out a minute        33060/tcp, 33060/tcp
wordpress-web        wordpress:latest     "docker-entrypoint.s...  wordpress  About a minute ago Up Ab
out a minute        0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
```

Compose 网络与数据卷

从截图可以看到，Docker Compose 为应用创建了独立网络，并创建了 `db_data` 数据卷用于保存 MySQL 数据。

## 4.8 验证数据库数据持久化

停止并删除 Compose 应用容器，但保留 volume：

```
docker compose down
```

重新启动应用：

```
docker compose up -d
```

查看 volume 是否仍然存在：

```
docker volume ls
```

实验结果如下图所示。

```
(base) PS D:\money\20260507\lab2\wordpress-demo> docker compose down
[+] Running 3/3
  ✓ Container wordpress-web          Removed          1.7s
  ✓ Container wordpress-db           Removed          1.5s
  ✓ Network wordpress-demo_default   Removed          0.6s
(base) PS D:\money\20260507\lab2\wordpress-demo> docker compose up -d
[+] Running 3/3
  ✓ Network wordpress-demo_default   Created          0.4s
  ✓ Container wordpress-db           Started          0.7s
  ✓ Container wordpress-web          Started          1.3s
(base) PS D:\money\20260507\lab2\wordpress-demo> docker volume ls
DRIVER              VOLUME NAME
local               1c510feb638c0bf1127fcd767190b12e49469bfcae2a79ef1c466e6722d91520
local               75e46eea850106f7a845b094293f2d96ab1070d55f778437e5a18e5886811c6e
local               b8ba9463061b5bc2cf17bde4f00b1998956302b83b64607227f66396d92d42ef
local               fdb1c1df0c736d200d5c8af2b50ad569908e787d7911470809307fc167e3b8
local               minikube
local               wordpress-demo_db_data
```

Compose 数据持久化验证

从截图可以看到，Compose 应用重启后，数据库 volume 仍然存在，说明数据库数据不会因为容器删除而丢失。

如果需要完全清理应用和数据，可以执行：

```
docker compose down -v
```

## 五、实验结果分析

本次实验完成了 Docker 容器运行基础操作，包括端口发布、覆盖默认配置、数据持久化、本地文件共享和多容器应用部署。

端口发布实验表明，容器内部端口需要通过 `-p HOST_PORT:CONTAINER_PORT` 映射到宿主机后，才能从本机浏览器访问。覆盖默认配置实验表明，可以通过环境变量和命令覆盖等方式修改容器运行行为。数据持久化实验表明，Docker volume 可以让数据在容器删除后继续保留。bind mount 实验表明，宿主机目录可以直接挂载到容器中，适合开发场景下共享代码或配置。多容器应用实验表明，Docker Compose 可以统一定义和启动多个相关服务，并自动创建网络、volume 和服务依赖关系。

整体来看，Docker 不只是简单地运行单个容器，还可以通过端口、环境变量、volume、bind mount 和 Compose 等机制构建完整的本地开发环境。

## 六、实验中遇到的问题与解决方法

### 6.1 Docker Engine 未启动

如果执行 Docker 命令时出现类似错误：

```
open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified
```

说明 Docker Desktop 的 Linux Engine 尚未启动。

解决方法：

1. 打开 Docker Desktop。
2. 等待 Docker Engine running。
3. 确认当前使用 Linux containers。
4. 重新执行：

```
docker version
```

### 6.2 端口被占用

如果启动 nginx、WordPress 等容器时提示端口被占用，可以更换宿主机端口。例如将：

```
-p 8080:80
```

改为：

```
-p 8088:80
```

浏览器访问地址也相应改为:

```
http://localhost:8088
```

## 6.3 镜像拉取失败

如果 Docker 拉取镜像失败, 可以检查网络和代理设置。确认 Docker Desktop 已经配置代理, 或者切换网络后重新执行:

```
docker pull nginx
```

## 6.4 Volume 无法删除

如果 volume 正在被容器使用, 直接删除会失败。需要先删除使用该 volume 的容器。

```
docker ps -a
docker rm -f 容器名
docker volume rm 数据卷名
```

## 6.5 Compose 应用没有完全清理

`docker compose down` 默认会删除容器和网络, 但不会删除 volume。如果需要删除 volume, 需要执行:

```
docker compose down -v
```

# 七、清理实验资源

实验完成后, 可以清理本次实验创建的容器、网络 and volume。

清理 nginx 容器:

```
docker rm -f web-demo bind-nginx
```

清理 Compose 应用:

```
cd D:\money\20260507\lab2\wordpress-demo
docker compose down -v
```

清理不用的镜像和悬空资源:

```
docker image prune
docker container prune
docker volume prune
```

如果需要查看 Docker 占用空间:

```
docker system df
```

## 八、实验总结

本次实验按照 Docker 官方教程完成了容器运行基础操作。通过 nginx 容器验证了端口发布，通过 Postgres 和 Alpine 容器验证了覆盖默认配置，通过 Docker volume 验证了数据持久化，通过 bind mount 验证了宿主机文件共享，通过 Docker Compose 部署了 WordPress 和 MySQL 多容器应用。

通过本次实验，可以得到以下结论：

1. Docker 容器默认具有隔离性，需要通过端口发布对外提供服务。
2. `docker run` 可以通过环境变量、命令覆盖、资源限制等参数改变容器行为。
3. Docker volume 适合保存需要长期保留的数据。
4. bind mount 适合开发场景下共享宿主机代码和配置文件。
5. Docker Compose 适合管理多个相互依赖的容器服务。
6. 使用 Docker Desktop 可以在 Windows 环境中方便地进行容器实验。

整体来看，本次实验建立了 Docker 容器运行和多容器应用管理的基本操作框架，为后续部署 Gitea、Kubernetes 和 Istio 等实验环境打下了基础。

## 九、参考资料

1. Docker Publishing and Exposing Ports  
<https://docs.docker.com/get-started/docker-concepts/running-containers/publishing-ports/>
2. Docker Overriding Container Defaults  
<https://docs.docker.com/get-started/docker-concepts/running-containers/overriding-container-defaults/>
3. Docker Persisting Container Data  
<https://docs.docker.com/get-started/docker-concepts/running-containers/persisting-container-data/>
4. Docker Sharing Local Files with Containers  
<https://docs.docker.com/get-started/docker-concepts/running-containers/sharing-local-files/>
5. Docker Multi-container Applications  
<https://docs.docker.com/get-started/docker-concepts/running-containers/multi-container-applications/>
6. Docker Desktop WSL 2  
<https://docs.docker.com/desktop/features/wsl/>